

Procedural Music System — Modification Guide

File Map

```
Scripts/  
  Core/TonalPitchSpace.cs      - Music theory (scales, chords, tension math)  
  Composition/CompositionEngine.cs - Decides WHAT notes to play and WHEN  
  Synthesis/SynthEngine.cs     - Generates actual AUDIO from notes  
  Synthesis/VoiceManager.cs    - Instrument presets and voice pooling  
  Bridge/ProceduralMusicController.cs - Unity MonoBehaviour, game-facing API  
  Examples/FullSystemTest.cs   - IMGUI test harness  
  Examples/MusicTestController.cs - Keyboard shortcut controller
```

Common Modifications

Change instrument volumes

File: `VoiceManager.cs` — the `InstrumentPreset` static properties at the top.

Each preset has a `Volume` field (0-1). Current values:

```
HurdyGurdy (Drone): 0.22
Flute (LegatoMelody): 0.3
Bass: 0.38
Log Drum (Kick): 0.85
Frame Drum (Snare): 0.45
Brush (HiHat): 0.18
Strings: 0.3
Kantele: 0.7
```

Just change the number. Higher = louder. The soft clipper in MasterMixer prevents distortion if things get too loud, but try to keep individual instruments under 0.9.

Change when instruments enter (layer thresholds)

File: `CompositionEngine.cs` — inside `GenerateMeasure()`, near the top (around line 808).

```
csharp

bool padActive = EnablePad;           // Always on
bool kanteleActive = EnableKantele;   // Always on
bool melodyActive = EnableMelody && lt >= 0.15f; // Flute at 0.15
bool stringsActive = EnableStrings && lt >= 0.3f; // Strings at 0.3
bool bassActive = EnableBass && lt >= 0.4f;      // Bass at 0.4
bool percActive = EnablePercussion && lt >= 0.55f; // Percussion at 0.55
```

`lt` is the unclamped layer tension (0-1). Change the threshold numbers to control when each layer enters. Setting to `0f` means "always on if the state enables it". Removing the `&& lt >= X` part does the same thing.

Note: these thresholds use `lt` (LayerTension), which is NOT affected by MaxTension. MaxTension only affects chord selection.

Change game state parameters

File: `ProceduralMusicController.cs` — inside `MusicStateConfig.GetDefault()`.

Each state has:

Field	What it does
<code>PreferredMode</code>	Scale/mode (Major, NaturalMinor, HarmonicMinor, Dorian, Mixolydian, Aeolian, Phrygian, Locrian)
<code>TempoMin</code> / <code>TempoMax</code>	BPM range. Tempo interpolates between these based on game tension
<code>BaseTensionOffset</code>	Added to game tension. Positive = more tense, negative = calmer
<code>MaxTension</code>	Ceiling for HARMONIC tension only (chord selection). Does NOT affect layer entry
<code>RootOffset</code>	Semitones from StartingKey. 0 = same key, 7 = up a fifth, -3 = down a minor third
<code>Pads</code> through <code>Kantele</code>	Master switches for each layer. If false, that layer never plays in this state
<code>FModIndexMultiplier</code>	Timbral brightness multiplier

Example — making Combat use Phrygian mode at a slower tempo:

csharp

```
case GameMusicState.Combat:
    return new MusicStateConfig
    {
        State = state, PreferredMode = MusicalMode.Phyrgian, // Changed
        TempoMin = 70, TempoMax = 95, // Slower
        BaseTensionOffset = 0.2f, MaxTension = 0.65f, RootOffset = -3,
        Pads = true, Melody = true, Bass = true, Percussion = true,
        Strings = true, Kantele = false, FMModIndexMultiplier = 1.5f
    };
```

Add a new game state

1. Add to the enum in `ProceduralMusicController.cs`:

csharp

```
public enum GameMusicState
{
    Explore, Dialogue, Tension, Combat, Victory, Mystery, Ambient, Spooky,
    YourNewState // Add here
}
```

2. Add a config case in `MusicStateConfig.GetDefault()`:

csharp

```

case GameMusicState.YourNewState:
    return new MusicStateConfig
    {
        State = state, PreferredMode = MusicalMode.Dorian,
        TempoMin = 80, TempoMax = 100,
        BaseTensionOffset = 0f, MaxTension = 0.7f, RootOffset = 0,
        Pads = true, Melody = true, Bass = false, Percussion = false,
        Strings = true, Kantele = true
    };

```

3. It will automatically appear in the test GUI state buttons and scenario dropdown.

4. Trigger it from game code: `music.SetGameState(GameMusicState.YourNewState);`

Change how the flute melody behaves

File: `CompositionEngine.cs` — `MelodyGenerator` class.

Settings (top of the class):

```

csharp

public int OctaveMin = 4;           // Lowest octave
public int OctaveMax = 4;           // Highest octave (same = 1 octave range)
public float RestProbability = 0.15f; // Chance of resting instead of playing
public float StepwiseMotionBias = 1.5f; // How much to prefer small intervals
public float SyncopationAmount = 0.12f; // Slight timing shifts
public float TiedNoteProbability = 0.2f; // Chance of holding a note longer
public float LeapRecoveryBias = 0.9f; // After a big jump, prefer stepping bac

```

How free the melody is (chordStrictness): In `BuildCandidates()`, the `chordStrictness` controls how tightly the flute follows chord tones:

```
csharp  
  
float chordStrictness = Mathf.Clamp01((tension - 0.2f) / 0.5f);
```

This means:

- Below tension 0.2: completely free (scale tones weight 4, chord tones weight 9)
- Above tension 0.7: locked to chord tones (scale tones weight 0.5, chord tones weight 20)
- Between: gradual transition

To make the melody always free: `float chordStrictness = 0f;` To make it always strict: `float chordStrictness = 1f;`

Rhythm patterns: In `SelectRhythmPattern()`, three pools of beat-offset patterns are selected by tension:

- Low (<0.3): long notes, half notes, sustained singles
- Medium (0.3-0.6): quarters, dotted, triplet, offbeat
- High (>0.6): syncopated, eighth notes, runs

The patterns themselves are defined in `_rhythmPatterns` at the top of the class. Each is an array of beat positions within a 4-beat measure.

Change how the kantele plays

File: `CompositionEngine.cs` — inside `GenerateMeasure()`, the kantele section (search for

`kanteleActive`)).

Three modes based on tension:

- `t < 0.3f`: sparse plucks at beats 0, 1.5, 3
- `t < 0.6f`: flowing eighth-note arpeggios through chord tones
- `t >= 0.6f`: rapid tremolo-picking on root and fifth

To change the arpeggio speed, modify the `spacing` variable in each section. To change which notes it plays, modify how `kantelePitches` is built.

Change instrument sounds

File: `SynthEngine.cs` — look at the synth type you want to modify.

Synth types and what they control:

Type	Used by	Key parameters
HurdyGurdy	Drone pad	Square wave melody + drone + constant buzz noise. Filter cutoff controls darkness. <code>_hurdyWobble</code> Rate/Depth controls pitch instability.
LegatoLead	Flute	Additive sine partials (harmonic volumes in <code>GenerateLegatoLead</code>). <code>_fluteBreathAmount</code> controls airiness. Vibrato fades in over time.
Subtractive	Bass	Saw + sub-sine through low-pass filter. <code>FilterCutoff</code> and <code>FilterEnvAmount</code> in the preset control brightness.
Percussion	All drums	Sine + noise, different per MIDI note. Change parameters in the NoteOn switch: pitch sweep range, decay rate, sine/noise ratio, noise filter cutoff.
StringEnsemble	Strings	4 detuned saws + filter + vibrato. Detune amounts in NoteOn. Filter cutoff in preset.
PluckedString	Kantele	Karplus-Strong delay buffer. <code>_pluckDamping</code> controls sustain (higher = longer ring). Preset <code>FilterCutoff</code> controls brightness.

To change the flute's harmonic recipe, edit `GenerateLegatoLead()`:

```
csharp
```



```
float sample = _flutePartials[0].NextSample() * 1.0f    // Fundamental
              + _flutePartials[1].NextSample() * 0.12f  // 2nd harmonic
              + _flutePartials[2].NextSample() * 0.18f  // 3rd harmonic
              + _flutePartials[3].NextSample() * 0.05f; // 4th harmonic
```

More 2nd/4th harmonic = warmer. More 3rd = nasal/reedy. Less of everything = purer.

To change the hurdy-gurdy buzz amount, edit `GenerateHurdyGurdy()`:

```
csharp
buzz = _hurdyBuzzFilter.Process(buzz) * 0.2f; // Change 0.2 for more/less grin
```

To change drum sounds, edit the Percussion NoteOn cases. The key variables per drum:

```
_percPitchEnv      - Starting frequency of the pitch sweep
_percPitchTarget   - Where the sweep lands
_percPitchDecay    - How fast it sweeps (closer to 1.0 = slower)
_percSineAmt       - How much tonal body
_percNoiseAmt      - How much noise/texture
_percNoiseFilter.Cutoff - Brightness of the noise
```

Add a new instrument

1. **Choose or create a synth type** in `SynthEngine.cs`:

- If an existing type works, skip to step 2
- Otherwise add to the `SynthType` enum, add fields, add NoteOn case, add NextSample case

2. **Create a preset** in `VoiceManager.cs`:

```
csharp

public static InstrumentPreset MyInstrument => new InstrumentPreset
{
    Name = "MyInstrument",
    SynthType = SynthVoice.SynthType.PluckedString, // or whatever type
    MaxPolyphony = 4,
    Attack = 0.01f, Decay = 1.0f, Sustain = 0f, Release = 0.5f,
    FilterCutoff = 3000f, FilterEnvAmount = 0f,
    Volume = 0.5f
};
```

3. **Register it** in `ProceduralMusicController.cs` `Initialize()`:

```
csharp

_myVoices = _mixer.AddInstrument(InstrumentPreset.MyInstrument); // index 8
```

Add the field: `private VoiceManager _myVoices;`

4. **Add an index** in `CompositionEngine.cs`:

```
csharp

public int MyInstrumentIndex = 8;
public bool EnableMyInstrument = true;
```

5. **Generate notes for it** in `GenerateMeasure()`:

csharp

```
if (EnableMyInstrument && lt >= 0.3f) // Enters at tension 0.3
{
    // Your note generation logic here
    _pendingNoteOns.Add((measureStart, new NoteEvent(midiNote, velocity, durati
}
```

6. **Wire up state configs** — add `MyInstrument = true/false` to `MusicStateConfig` and each state's default.
7. **Update the test GUI** — add the name to `_instrumentNames` array in `FullSystemTest.cs`.

Change the key at runtime

csharp

```
// Change root only (keeps current mode, updates StartingKey for future state c
music.SetKey(PitchClass.D);

// Change root and mode
music.ForceModulation(PitchClass.Fs, MusicalMode.HarmonicMinor);

// Change mode only (keeps current root)
music.ForceModulation(music.StartingKey, MusicalMode.Phyrgian);
```

Change how chords are chosen

File: `TonalPitchSpace.cs` — `SuggestNextChords()`.

This function returns the top 4 chord candidates ranked by how close their TPS tension matches the target. To change behavior:

- **More variety:** increase the candidate count or add non-diatonic chords
 - **More stable:** decrease `targetTension` in the bridge or lower `MaxTension`
 - **Secondary dominants:** these are added when tension > 0.6 (search for "secondary dominants" in the file)
-

Change the reverb

File: `VoiceManager.cs` — `MasterMixer.FillBuffer()`.

Current reverb: simple comb filter with 50ms delay, 0.4 feedback, 0.15 wet mix.

```
csharp

private float _reverbDelay = 0.05f;      // Delay time in seconds
private float _reverbFeedback = 0.4f;    // How much signal feeds back (0-0.95)
private float _reverbMix = 0.15f;        // Wet/dry mix (0 = dry, 1 = all reverb)
```

For a larger space: increase delay to 0.08-0.1, feedback to 0.5-0.6.

For a tighter sound: decrease delay to 0.02-0.03, feedback to 0.2-0.3.

Tension flow diagram

Game Code

|

└─ SetTension(0.8)

|

└─ Bridge (ProceduralMusicController)

|

└─ effectiveTension = 0.8 + BaseTensionOffset

|

└─ TensionTarget = min(effectiveTension, MaxTension) → Chord

selection

|

musical)

|

└─ LayerTension = effectiveTension → Layer entry thresholds
(unclamped, all instruments

can enter)

(clamped,